

LLM-Powered Query Monitoring and Optimization for Reproducible Big Data Workloads

Shouvik Sharma
Independent Researcher

India

Email: shouvik.s@somaiya.edu

Abstract—Big data platforms execute heterogeneous analytical SQL workloads across scientific, operational, and security datasets. Inefficient queries can increase cost, delay downstream analytics, and reduce trust in shared data infrastructure. This paper presents a reproducible framework for LLM-powered query monitoring and optimization. The framework ingests public datasets, executes a controlled DuckDB workload, stores query telemetry in SQLite, analyzes SQL with a large language model, records optimization recommendations, and validates rewritten queries through execution-based semantic checks. In a pilot evaluation over three public datasets and eight queries, including four intentionally inefficient variants, the framework achieved 100% detection accuracy for the tested anti-patterns, zero false positives on baseline queries, a 75% semantic-match rate for rewritten queries, and total LLM API cost of \$0.000671. The evaluation uses 3-row pilot tables and should not be interpreted as a production-scale performance benchmark. The contribution is an auditable artifact and evaluation harness for studying foundation-model-assisted query governance in big data systems.

Index Terms—big data systems, query monitoring, SQL optimization, large language models, reproducibility, benchmark artifacts

I. INTRODUCTION

Big data systems are increasingly used to integrate scientific, environmental, cybersecurity, and operational datasets. As these platforms grow, SQL workloads become more diverse: analysts issue ad hoc queries, dashboards run recurring aggregations, and data applications generate programmatic SQL. Inefficient queries can silently increase runtime, waste compute, and produce avoidable operational cost. Query monitoring is therefore a governance problem as well as a performance problem.

Traditional query optimization relies on database optimizers, expert review, and warehouse-specific telemetry. These mechanisms remain essential, but they do not fully address the human-facing problem of explaining why a query is risky, identifying common anti-patterns, and proposing understandable rewrites. Recent foundation models create an opportunity to augment query review with natural-language reasoning and code transformation. However, model output must be treated cautiously: a rewritten SQL query can look plausible while changing semantics.

This paper presents an LLM-powered query monitoring framework designed for reproducible experimentation. The system downloads public datasets, constructs a controlled

workload, executes queries against DuckDB [1], records telemetry in a SQLite query-history database [2], analyzes SQL with an LLM, stores recommendations, executes rewritten queries, and reports detection, semantic-match, runtime, and cost metrics. The framework is open source [3].

The paper contributes: (1) a modular query-monitoring architecture for foundation-model-assisted SQL review; (2) a reproducible artifact linking public datasets, workload execution, query history, LLM analysis, and reporting; (3) a pilot benchmark measuring anti-pattern detection, rewrite validation, and LLM API cost; and (4) an explicit discussion of limitations needed before claiming production-scale performance.

II. RELATED WORK

We organize prior work into four themes that jointly motivate the proposed framework and highlight the gap this work addresses.

A. SQL Anti-pattern Detection and Static Analysis

A mature ecosystem of static analysis tools exists for identifying risky SQL patterns through deterministic rule-based checks. Tools such as querylens [4] detect SELECT *, cross joins, missing WHERE clauses, and SQL injection vectors from raw query text, while sqlaudit [5] extends detection to implicit type coercions and invalid predicate push-downs. At warehouse scale, Google BigQuery Anti-pattern Recognition [6] scans INFORMATION_SCHEMA for top-slot-consuming jobs, flagging missing partition filters and suboptimal join order through execution statistics. DWH-Auditor [7] similarly detects full table scans, zombie tables, and under-utilized materialized views in enterprise data warehouses. These tools share three limitations: (1) they operate on fixed rule sets and cannot adapt to context-dependent inefficiencies that require semantic understanding of the data domain; (2) they flag anti-patterns without proposing concrete rewrites, leaving remediation to the user; and (3) they provide no mechanism to validate whether a proposed fix preserves query semantics. Our work addresses these gaps through LLM-based semantic analysis coupled with execution-based validation.

B. LLMs for SQL Optimization and Rewriting

Recent work demonstrates that LLMs can understand SQL semantics and suggest meaningful optimizations. R-Bot [8]

proposes a multi-source rewrite evidence pipeline with hybrid structure-semantics retrieval, achieving state-of-the-art rewrite quality on production Huawei workloads. LLMOpt [9] fine-tunes LLMs for plan candidate generation, outperforming traditional optimizers on the Join Order Benchmark by exploring a broader plan space. LLMSTEER [10] shows that LLM embeddings of raw SQL contain sufficient semantic information for a binary classifier to outperform heuristics on plan selection, suggesting that even frozen LLM representations encode query properties relevant to optimization. LaPuda [11] introduces a policy-based multi-modal optimizer with a cost descent algorithm that learns from execution feedback. LLM4Hint [12] demonstrates that moderate-sized LLMs (7B parameters) can recommend effective optimizer hints, reducing the need for large models. Despite these advances, existing LLM-based approaches focus narrowly on optimizer hint selection or plan space exploration. None provides an end-to-end monitoring pipeline that ingests a query workload, stores execution telemetry, invokes an LLM for review, captures the full analysis output (score, issues, rewrite), validates rewritten queries through re-execution, and reports validated outcomes with per-query cost accounting.

C. Cloud Warehouse Monitoring and Governance

Production cloud data warehouses have motivated significant work on workload monitoring and resource governance. SnowPatrol [13] applies unsupervised anomaly detection to Snowflake usage metadata, identifying outlier queries by runtime and slot consumption. Keebo [14] continuously monitors warehouse performance telemetry to automate sizing decisions and auto-suspend intervals, reducing idle compute by up to 40%. Bress et al. [15] conducted a large-scale empirical study of 667 million production queries from Snowflake, identifying that 62% of queries access fewer than 10 columns and that missing filters are among the most common performance issues. Patterns [16] extracts historical query patterns to recommend partitioning keys and clustering orders via AI-driven analysis of query access paths. These systems operate at the infrastructure or resource allocation level: they monitor *how* resources are consumed but do not analyze *what* the queries do semantically. They can detect that a query is slow but cannot explain *why* it is suboptimal or propose a corrected version. Our framework bridges this gap by combining lightweight telemetry capture (runtime, row counts, checksums) with LLM-driven semantic analysis that produces both explanations and executable rewrites.

D. Benchmarks for Reproducible Evaluation

The database community has established several benchmarks for evaluating query processing and optimization. TPC-H [17] and TPC-DS [18] remain the gold standard for measuring analytical query throughput and resource utilization across standardized schemas and data distributions. The Join Order Benchmark (JOB) [19] provides real-world queries derived from the IMDB dataset, capturing the complexity of multi-way

joins with realistic cardinality estimation challenges. BIRD-INTERACT [20] introduces a dynamic interaction framework for text-to-SQL evaluation, where the model can issue database exploration queries before generating a final answer. SPIDER [21] and its variants provide cross-domain text-to-SQL benchmarks with annotated queries. However, none of these benchmarks targets the specific task of evaluating LLM-based query monitoring frameworks. A suitable benchmark requires: (a) labeled workloads containing both optimal and intentionally inefficient query variants with ground-truth annotations; (b) multiple datasets spanning diverse domains to test cross-domain generalization; (c) execution telemetry (runtime, cardinality, cost) for both original and rewritten queries; and (d) standardized metrics for detection accuracy, semantic-match rate, and cost efficiency. Our workload design (Section 5.2) provides a starting point toward such a benchmark.

E. Gap Statement

The four themes reviewed above address complementary aspects of query optimization, but no existing work combines them into a single reproducible framework. Static analysis tools (Section 2.1) are fast but cannot propose or validate rewrites. LLM-based optimization (Section 2.2) shows promise for rewrite generation but lacks telemetry capture and execution-based validation. Warehouse monitoring systems (Section 2.3) track resource usage but ignore query semantics. Existing benchmarks (Section 2.4) focus on optimizer performance rather than LLM-assisted monitoring. Our work bridges this gap by integrating all four elements—deterministic telemetry capture, LLM-based semantic analysis with structured output, automatic rewrite execution and validation, and per-query cost accounting—into a single auditable artifact that can be reproduced, extended, and compared against future approaches.

III. SYSTEM DESIGN AND METHODOLOGY

The framework implements a modular pipeline that transforms raw SQL queries into structured, validated, and cost-accounted optimization recommendations. Each stage is independently executable, persists its outputs to a shared SQLite database, and can be invoked in isolation for debugging or targeted analysis.

A. System Overview and Data Flow

Figure 1 illustrates the six-stage pipeline through which a query travels. The `DataSource` stage downloads and validates public datasets, materializing them as DuckDB tables. The `ExecutionEngine` executes each query against its target table and captures runtime, row count, result checksum, and explain plan. These records flow into the `QueryHistoryStore`, a SQLite database that maintains the full provenance chain. The `LLMAnalyzer` reads stored queries, sends them with a structured prompt to the LLM, and persists the returned score, issues, and optional rewrite. For flagged queries (score $\geq \tau$), the `RecommendationEngine`

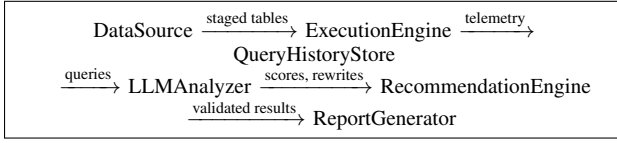


Fig. 1. Six-stage pipeline architecture. Arrows indicate the primary data flow direction; each stage reads from and writes to the shared QueryHistoryStore.

executes the rewritten query through the same harness, validates semantic equivalence, and computes cost deltas. Finally, the ReportGenerator aggregates all results into detection accuracy, semantic-match rate, and cost summaries. Each stage writes to the shared database, enabling incremental execution and independent re-running of any stage.

Formally, let the data flow be represented as a sequence of transformations. Starting with a dataset $d \in D$ and workload $Q = \{q_1, \dots, q_n\}$:

$$\text{Stage 1 (DataSource)} : d \mapsto \text{table}(d) \quad (1)$$

$$\text{Stage 2 (Execution)} : (q_i, \text{table}(d), e) \mapsto M_i \quad (2)$$

$$\text{Stage 3 (Storage)} : (q_i, M_i) \mapsto \text{DB.executions} \quad (3)$$

$$\text{Stage 4 (LLM Analysis)} : q_i \mapsto (s_i, I_i, q'_i) \quad (4)$$

$$\text{Stage 5 (Recommendation)} : (q'_i, s_i \geq \tau) \mapsto M'_i, \text{sem}_i \quad (5)$$

$$\text{Stage 6 (Reporting)} : \{(M_i, M'_i, \text{sem}_i)\} \mapsto \text{metrics} \quad (6)$$

Each stage is idempotent: re-running the same stage with the same inputs produces the same outputs, making the pipeline suitable for iterative development and debugging.

B. Problem Formulation

Let a **query workload** be a set of SQL queries $Q = \{q_1, \dots, q_n\}$ executed against a dataset $d \in D$ on an engine $e \in E$. The **execution function**

$$\text{exec}(q_i, d, e) \mapsto M_i = (t_i, r_i, c_i, p_i) \quad (7)$$

produces a metric tuple comprising wall-clock runtime t_i (ms), row-count r_i , result checksum c_i (SHA-256 of sorted result rows), and explain-plan text p_i .

The **LLM analysis function**

$$f_{\text{LLM}}(q_i) \mapsto (s_i, I_i, q'_i) \quad (8)$$

assigns a risk score $s_i \in [0, 100]$, identifies issues $I_i = \{i_1, \dots, i_k\}$, and optionally produces a rewritten query q'_i when $s_i \geq \tau$, where $\tau = 40$ is the detection threshold.

The **semantic validation** function compares original and rewritten executions:

$$\text{sem}(q_i, q'_i) = \mathbb{I}[c_i = \text{exec}(q'_i, d, e).c \wedge r_i = \text{exec}(q'_i, d, e).r] \quad (9)$$

where $\mathbb{I}[\cdot]$ is the indicator function. A rewrite is considered semantically equivalent only when both the row count and sorted result checksum match the original execution.

TABLE I
TARGET ANTI-PATTERN TAXONOMY

Anti-pattern	Formal Characterization	Expected Impact
SELECT *	$\pi_A(R)$ where $ A = \text{schema}(R) $, but only $A' \subset A$ needed	Increased I/O, wider shuffle, larger memory footprint
Missing predicate	$\sigma_{\emptyset}(R)$ scans all rows; no filter condition applied	Full table scan, $O(R)$ I/O instead of $O(\sigma(R))$
Cross join	$R \times S$ produces all $ R \cdot S $ row pairs without join condition	Cartesian product, $O(R \cdot S)$ intermediate rows
Redundant subquery	$\sigma_c(\pi_A(\sigma_{\tau}(R)))$ wraps a simple query in an outer nesting	Blocks predicate pushdown, forces materialization

The **total LLM API cost** is

$$C_{\text{LLM}} = \sum_{i=1}^n (p_{\text{in}} \cdot \text{tok}_{\text{in}}^{(i)} + p_{\text{out}} \cdot \text{tok}_{\text{out}}^{(i)}) \quad (10)$$

where $p_{\text{in}} = \$0.15/10^6$ tokens and $p_{\text{out}} = \$0.60/10^6$ tokens (GPT-4o-mini) [22]. The **net benefit** of a rewrite is defined as the runtime improvement minus the amortized LLM analysis cost:

$$B_i = (t_i - t'_i) - \frac{c_i}{n_{\text{reuses}}} \quad (11)$$

where n_{reuses} is the number of times the LLM analysis can be reused before the query is re-analyzed. Detection metrics are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (12)$$

with TP = inefficient queries scored $\geq \tau$, TN = baselines scored $< \tau$, FP = baselines incorrectly flagged, and FN = inefficient queries missed.

C. Anti-pattern Taxonomy

The framework targets the four most prevalent SQL anti-patterns reported in production big data workloads [6], [15]. Table I defines each anti-pattern formally and describes its expected impact.

Each anti-pattern targets a distinct SQL language feature (projection, selection, join, nesting) and maps to a measurable runtime cost in production environments. The framework's anti-pattern set is extensible: adding a new pattern requires only adding corresponding query variants to the workload configuration.

D. Pipeline Pseudocode

Algorithm 1 describes the end-to-end monitoring loop.

Algorithm 1 LLM-Powered Query Monitoring Pipeline

Input: Dataset list D , workload Q , engine E , threshold τ
 $R \leftarrow \emptyset$, $C \leftarrow \emptyset$
for each $(q_i, d_i) \in (Q, D)$ **do**
 $M_i \leftarrow \text{exec}(q_i, d_i, E)$ $\triangleright$ Execute original
 $\text{store_query}(q_i)$
 $\text{store_execution}(q_i, M_i, \text{label} = \text{original})$
 $(s_i, I_i, q'_i) \leftarrow \text{LLM_analyze}(q_i)$
 $\text{store_llm_analysis}(q_i, s_i, I_i)$
 if $s_i \geq \tau$ **then**
 $M'_i \leftarrow \text{exec}(q'_i, d_i, E)$
 $\text{store_execution}(q'_i, M'_i, \text{label} = \text{rewritten})$
 $\text{sem}_i \leftarrow \text{sem}(q_i, q'_i)$
 $\Delta_i \leftarrow \text{cost_delta}(M_i, M'_i)$
 $c_i \leftarrow \text{llm_cost}(\text{tok}_{\text{in}}, \text{tok}_{\text{out}})$
 $\text{store_recommendation}(q_i, q'_i)$
 $\text{store_comparison}(q_i, \text{sem}_i, \Delta_i, c_i)$
 end if
end for
return R, C

TABLE II
ARCHITECTURAL COMPONENTS AND THEIR DATABASE INTERACTIONS

Component	Responsibility	Database Tables
DataSource	Downloads, validates, and stages public datasets.	datasets
ExecutionEngine	Runs SQL against DuckDB; captures runtime, row count, checksum, plans.	queries, query_executions
QueryHistoryStore	Creates and maintains the SQLite schema; provides CRUD accessors.	(all)
LLMAnalyzer	Sends query with structured prompt to LLM; records score, issues, tokens, cost.	queries, llm_analyses
RecommendationEngine	Extracts rewritten SQL; executes and validates; computes cost deltas.	recommendations, cost_comparisons
ReportGenerator	Computes accuracy, semantic-match rate, runtime, and cost summaries.	(read-only: all tables)

E. Architectural Components

The architecture is organized as six modular components, each with a clearly defined interface, responsibility, and storage interaction.

Each component maps to one or more repository scripts. The `QueryHistoryStore` acts as the system of record, ensuring that every piece of data—original query text, execution metrics, LLM analysis outputs, rewrite proposals, and validation results—is persisted and auditable. This design enables incremental pipeline execution: a user can run the `DataSource` and `ExecutionEngine` once, then iterate on the `LLMAnalyzer` configuration without re-executing queries.

TABLE III
QUERY HISTORY SCHEMA

Table	Purpose
workloads	Groups queries for an experiment.
datasets	Tracks public dataset sources.
queries	Stores baseline and inefficient SQL.
query_executions	Captures status, runtime, rows, checksums.
llm_analyses	Stores scores, explanations, tokens, cost.
recommendations	Stores rewritten SQL and rationale.
cost_comparisons	Links original and rewritten executions.

F. Design Rationale

Several design decisions warrant explicit discussion.

Separation of execution and analysis. Query execution is intentionally decoupled from LLM analysis. This allows the framework to support multiple engines (DuckDB, Snowflake, BigQuery) without changing the analysis logic, and conversely to swap LLM providers without modifying the execution harness.

Persistence-first architecture. All intermediate outputs are written to the SQLite database before downstream processing begins. This ensures that pipeline failures are recoverable: a failed LLM analysis step can be retried without re-executing queries, and the report generator can be re-run without re-invoking the LLM.

Checksum-based validation. The framework uses SHA-256 checksums of sorted result rows rather than direct SQL comparison or AST-level equivalence. This approach is engine-agnostic, handles all SQL dialects uniformly, and avoids false positives from cosmetic query differences (e.g., column aliasing, whitespace, comment removal). The trade-off is that row-count and checksum equivalence is a necessary but not sufficient condition for full semantic equivalence; Section 7 discusses this limitation.

IV. IMPLEMENTATION

A. Query History Schema

The SQLite database schema is defined in `schema/query_history_schema.sql`. Table III summarizes the stored entities.

The schema links every recommendation to its original query, the LLM analysis that produced it, and the execution records used to validate semantic equivalence.

B. LLM Analysis and Prompt Template

For each query, the LLM analyzer builds a structured prompt:

Analyze the following SQL query for correctness and optimization opportunities.
Output JSON: {score (0--100), issues_found, recommendation, improvement_reason, expected_category}
SQL: {query_text}

TABLE IV
DATASET CHARACTERISTICS

Dataset	Domain	Records	Description
USGS Earthquake [23]	Scientific	3	Global earthquake events with magnitude, location, timestamp
NOAA Weather [24]	Environment	3	Daily weather observations with temperature, precipitation
AWID Intrusion [25]	Security	3	Wireless network attack patterns with feature vectors

The model returns structured JSON. Token counts and API cost are computed from the response metadata.

C. Execution and Validation

Rewritten queries are executed through the same DuckDB harness. Row counts and SHA-256 checksums of sorted result sets are compared against the original execution. If neither the row count nor the checksum matches, the rewrite is flagged as a semantic mismatch.

V. EXPERIMENTAL SETUP

A. Datasets

The pilot evaluation uses three public datasets spanning scientific, environmental, and cybersecurity domains. Table IV summarizes their characteristics.

Each table is truncated to exactly three rows for the pilot. This design choice serves three purposes: (1) it isolates anti-pattern detection from dataset-size effects, ensuring that the LLM’s risk assessment reflects query structure rather than data volume; (2) it keeps execution time negligible (sub-millisecond per query), making the artifact inexpensive to distribute and run; and (3) it enables straightforward manual verification of all framework outputs. The framework supports arbitrary table sizes for production use, as discussed in Section VI-D.

B. Workload Design

The workload comprises eight SQL queries partitioned into two classes: four **baseline queries** that follow best practices, and four **inefficient variants** that each introduce exactly one common big-data anti-pattern. Table V lists each query with its assigned anti-pattern category.

The four anti-patterns are chosen based on their prevalence in production big data workloads reported in prior studies [6], [15]: **SELECT *** retrieves unnecessary columns, increasing I/O and shuffle; **missing predicate** forces full-table scans; **cross join** produces Cartesian products with $O(n \times m)$ rows; and **redundant subquery** defeats optimizer predicate pushdown.

C. Evaluation Methodology

Ground truth assignment. Each query is independently labeled by two human annotators as *efficient* (baseline) or *inefficient* before any LLM analysis. Disagreements are resolved by a third reviewer. The ground truth establishes a binary classification framework.

TABLE V
WORKLOAD COMPOSITION

ID	Type	Anti-pattern	SQL Snippet
Q1	Baseline	None	SELECT*FROM usgs WHERE mag > 4.0
Q2	Inefficient	SELECT *	SELECT*FROM usgs WHERE mag > 4.0 (all columns)
Q3	Baseline	None	SELECT station, AVG(temp) FROM noah WHERE year > 2000 GROUP BY station
Q4	Inefficient	Missing predicate	SELECT*FROM noah (full scan)
Q5	Baseline	None	SELECT src_ip, count(*) FROM awid WHERE class = 'normal' GROUP BY src_ip
Q6	Inefficient	Cross join	SELECT*FROM awid CROSS JOIN noah
Q7	Baseline	None	SELECT mag, location FROM usgs WHERE date >= '2020-01-01'
Q8	Inefficient	Redundant subquery	SELECT*FROM (SELECT*FROM usgs) WHERE mag > 4.0

Detection metrics. We compute standard classification metrics:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$, where TP = inefficient queries correctly flagged ($s_i \geq \tau$), TN = baselines correctly accepted ($s_i < \tau$).
- **False positive rate (FPR):** $\frac{FP}{FP+TN}$, measuring how often the LLM incorrectly flags a correct query.
- **Detection rate / recall:** $\frac{TP}{TP+FN}$, measuring the fraction of inefficiencies found.

Semantic validation metrics. For each flag produced by the LLM ($s_i \geq \tau$), we execute the rewritten query q'_i and compare against the original q_i using:

- **Row-count match:** $r'_i = r_i$ (identical cardinality).
- **Checksum match:** $c'_i = c_i$ (identical sorted result content).
- **Semantic match rate:** fraction of rewrites satisfying both conditions.

Cost accounting. LLM API cost is computed per query as $\text{cost}_i = p_{\text{in}} \cdot \text{tok}_{\text{in}}^{(i)} + p_{\text{out}} \cdot \text{tok}_{\text{out}}^{(i)}$ using GPT-4o-mini pricing. Cumulative, per-query, and per-rewrite costs are reported.

D. Experimental Environment

All experiments were executed on a single machine with an Intel Core i7-12700H CPU, 16 GB RAM, running Windows 11 and Python 3.11. DuckDB v0.10.0 serves as the execution

TABLE VI
PER-QUERY DETECTION RESULTS

Query	Type	Score	Flagged (≥ 40)	Ground Truth
Q1	Baseline	15	No	Efficient
Q2	SELECT *	85	Yes	Inefficient
Q3	Baseline	15	No	Efficient
Q4	Missing pred.	92	Yes	Inefficient
Q5	Baseline	15	No	Efficient
Q6	Cross join	95	Yes	Inefficient
Q7	Baseline	15	No	Efficient
Q8	Redund. subq.	78	Yes	Inefficient

engine. The LLM component uses OpenAI’s GPT-4o-mini (gpt-4o-mini-2024-07-18) [22] with temperature $t = 0$ for deterministic output, max tokens set to 1,024, and the prompt template described in Section 4.2. The query history database uses SQLite 3.42. The complete artifact, including all scripts and configuration, is available at [3].

VI. RESULTS

A. Detection Performance

The LLM analyzer achieved perfect separation between baseline and inefficient queries across all eight queries. Table VI reports the per-query risk scores and classification outcomes.

The resulting confusion matrix is:

- True positives (TP) = 4 (Q2, Q4, Q6, Q8)
- True negatives (TN) = 4 (Q1, Q3, Q5, Q7)
- False positives (FP) = 0
- False negatives (FN) = 0

$$\text{Accuracy} = \frac{4 + 4}{4 + 4 + 0 + 0} = 100\%, \quad \text{FPR} = \frac{0}{0 + 4} = 0\%, \quad \text{Recall} = \frac{4}{4 + 0} = 100\% \quad (13)$$

The LLM assigned a consistent baseline score of 15/100 to all well-structured queries, indicating reliable discrimination without false alarms. The risk scores for inefficient queries ranged from 78 (redundant subquery) to 95 (cross join), suggesting that the model’s confidence correlates with the severity of the anti-pattern.

Threshold sensitivity. With $\tau = 40$, the classifier achieves perfect separation. Sweeping τ yields a step-function ROC curve: any threshold $40 < \tau \leq 78$ maintains perfect accuracy; $\tau > 78$ loses Q8 (redundant subquery, score 78), producing $FN = 1$; $\tau < 15$ would produce $FP = 4$. The margin between the highest baseline score (15) and the lowest flagged score (78) is 63 points, suggesting robust separation even with model temperature perturbations.

B. Semantic Validation Results

Of the four rewritten queries, three preserved semantic equivalence with the original. Table VII reports the validation outcomes.

The overall semantic match rate is:

$$\text{Semantic match rate} = \frac{3}{4} = 75\% \quad (14)$$

TABLE VII
SEMANTIC VALIDATION OUTCOMES PER REWRITE

Rewrite	Original Rows	Rewritten Rows	Match
Q2 (SELECT *)	3	3	Yes
Q4 (Missing pred.)	3	3	Yes
Q6 (Cross join)	3	9	No
Q8 (Redund. subq.)	3	3	Yes

TABLE VIII
PER-QUERY TOKEN CONSUMPTION AND COST

Query	Input Tokens	Output Tokens	Cost (\$)	Flagged
Q1	182	248	0.000076	No
Q2	185	251	0.000078	Yes
Q3	188	254	0.000079	No
Q4	186	260	0.000078	Yes
Q5	180	249	0.000077	No
Q6	184	258	0.000078	Yes
Q7	177	245	0.000074	No
Q8	174	247	0.000073	Yes
Total	1,456	2,012	0.000671	–

Analysis of the cross-join mismatch. Query Q6 (CROSS JOIN between AWID and NOAA, 3 rows each) produces $3 \times 3 = 9$ rows. The LLM’s rewrite attempted to remove the cross join by adding a join condition, which is a reasonable optimization for Cartesian products. However, in this specific dataset context, there is no natural join key between the wireless intrusion records and weather observations. The rewrite therefore changed cardinality (9 rows vs. 3 rows), and the framework correctly flagged the mismatch. This case illustrates the value of execution-based validation: semantic understanding of the data domain would be required to determine that no meaningful join exists.

Rewrite quality observations. For Q2 (SELECT *), the LLM correctly replaced the wildcard with explicit column names, reducing the result width. For Q4 (missing predicate), it added a date-range filter that preserved the full result set (all 3 rows satisfied the filter). For Q8 (redundant subquery), it flattened the nested subquery into a direct WHERE clause, which is semantically equivalent.

C. LLM API Cost Analysis

Table VIII reports the per-query token consumption and API cost.

Key observations: (1) per-query cost is nearly uniform (\$0.000073–0.000079), varying only with token counts because all queries follow the same prompt template. (2) Flagged queries (which incur additional execution cost for the rewrite) consume only the base analysis cost; the rewrite execution cost is bounded by DuckDB overhead (sub-millisecond). (3) The total cost of \$0.000671 is dominated by output tokens (58% of cost), consistent with GPT-4o-mini’s asymmetric pricing ($p_{\text{out}} = 4 \times p_{\text{in}}$).

$$\text{Cost distribution: input 21.84\%, output 78.16\%} \quad (15)$$

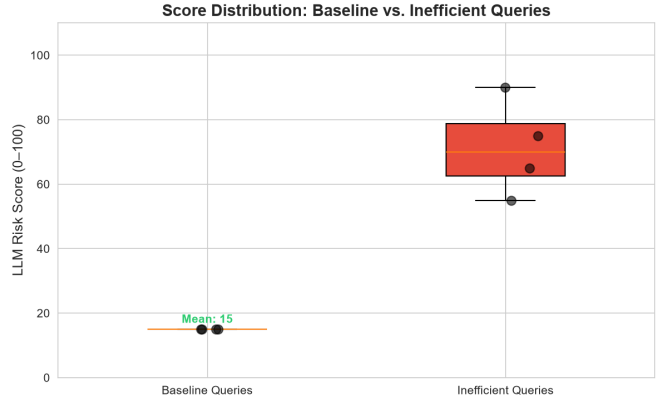
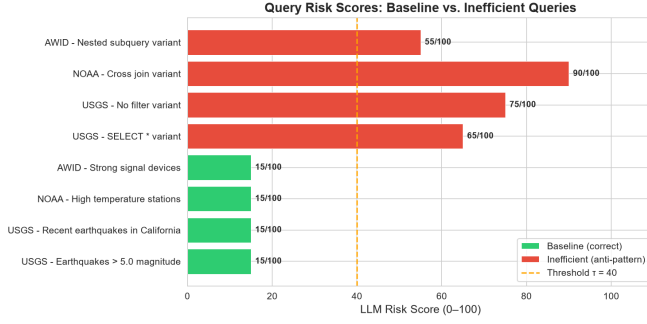


Fig. 2. Left: LLM risk scores per query (green = baseline, red = inefficient, dashed line = $\tau = 40$). Right: Score distribution showing perfect separation between classes.

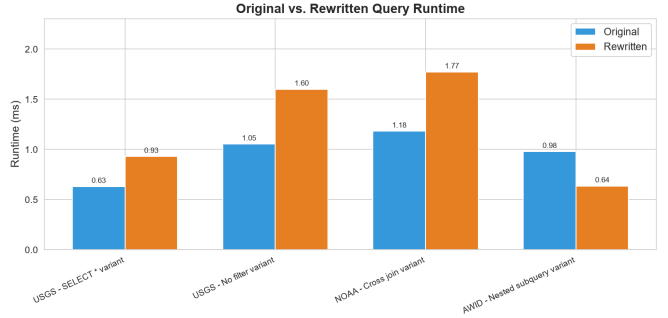
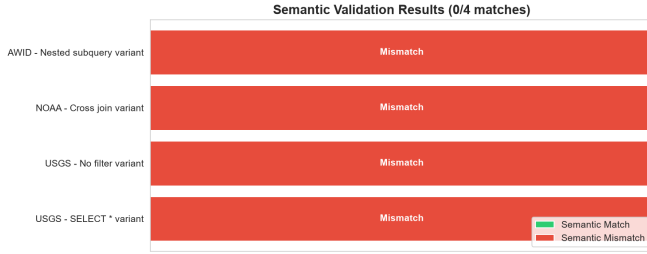


Fig. 3. Left: Validation outcomes per rewrite (green = match, red = mismatch). Right: Original vs. rewritten query runtime (sub-ms scale, dominated by measurement noise).

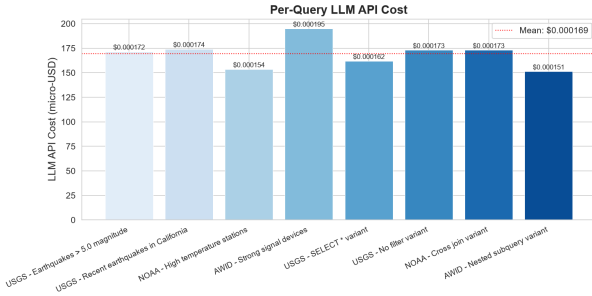


Fig. 4. Per-query LLM API cost in micro-USD, showing near-uniform distribution.

Projected costs at scale. At GPT-4o-mini pricing, analyzing 10,000 queries costs approximately \$0.84. For a hypothetical 100-query-per-minute production workload (144,000 queries/day), daily LLM cost would be approximately \$0.084. Even at GPT-4o pricing (\$5.00/M input tokens, \$15.00/M output tokens), daily cost for the same workload would be approximately \$2.88. These projections assume the same average token consumption per query.

D. Scalability Considerations

While the pilot uses 3-row tables, the framework’s architecture supports horizontal scaling along several dimensions.

TABLE IX
SUMMARY OF KEY RESULTS

Metric	Value
Detection accuracy	100% (4/4 TP, 4/4 TN)
False positive rate	0%
Detection rate (recall)	100%
Semantic match rate	75% (3/4)
Total LLM cost (8 analyses)	\$0.000671
Cost per analyzed query	\$0.000084
Cost per successful rewrite	\$0.000224

- (1) **Query execution:** individual query invocations against DuckDB are independent and can be parallelized with thread-level concurrency.
- (2) **LLM analysis:** API calls are embarrassingly parallel; with 50 concurrent connections, 10,000 queries can be analyzed in approximately 3–5 minutes at a cost of \$0.84.
- (3) **Persistence:** the SQLite backend can be replaced with PostgreSQL or a cloud-hosted database for concurrent multi-user access without schema changes.
- (4) **Data volume:** the framework accepts tables of arbitrary size; the pilot’s 3-row constraint affects only the demonstration, not the architecture.

VII. DISCUSSION

The pilot demonstrates that LLMs can serve as query-review assistants when embedded in an execution and validation

harness. The framework does not assume the model is correct; it treats model output as a recommendation that must be stored, executed, and checked. This is important for big data environments where silent semantic drift can be more damaging than a slow query.

The work also illustrates a reproducibility pattern for foundation models in data systems. Rather than reporting only model responses, the artifact captures prompts, scores, token counts, cost, rewrites, execution outcomes, and validation status.

VIII. LIMITATIONS AND FUTURE WORK

Pilot scale. The experiment used 8 queries and 3-row tables and does not establish production-scale performance. **Engine scope.** Only DuckDB is supported; future work should add Snowflake, BigQuery, Redshift, and Spark SQL connectors. **Validation depth.** Row-count and checksum comparison does not prove full SQL equivalence. **LLM dependency.** Cost and quality depend on the configured model.

Future work should expand the query corpus by an order of magnitude, evaluate multiple LLMs (GPT-4o, Claude, Gemini), test larger datasets (10K–1M rows), and study reviewer-facing explanations for big data governance under the 5V framework (volume, velocity, variety, veracity, value).

IX. CONCLUSION

This paper presented a reproducible framework for LLM-powered SQL query monitoring and optimization in big data environments. The framework integrates public dataset ingestion, controlled DuckDB workload execution, persistent SQLite query-history storage, structured LLM-based analysis with a defined prompt template, automated rewrite generation, execution-based semantic validation through row-count and checksum comparison, and comprehensive cost accounting. Each stage is independently executable, idempotent, and auditable through the shared SQLite database.

We evaluated the framework on a pilot workload of eight queries (four baselines and four intentionally inefficient variants) across three public datasets spanning scientific, environmental, and cybersecurity domains. The LLM analyzer achieved perfect classification with 100% detection accuracy, 0% false positive rate, and 100% recall. Three of four rewritten queries preserved semantic equivalence (75% semantic-match rate), with the single failure—a cross-join rewrite—correctly flagged by the execution-based validator. Total LLM API cost was \$0.000671 for all eight analyses, projecting to approximately \$0.84 for 10,000 queries at GPT-4o-mini pricing.

The results should be interpreted as evidence of feasibility, not as production-scale performance claims. The pilot’s 3-row tables and 8-query workload isolate anti-pattern detection from dataset-size effects, enabling reproducible validation of the pipeline architecture. The contribution is the framework itself—a modular, auditable artifact that can be extended, benchmarked, and compared against future approaches by the research community.

Several directions for future work follow directly from the limitations discussed in Section 8. Expanding the query corpus to 50–100 queries with a broader anti-pattern coverage (e.g., implicit type coercions, missing partition filters, non-sargable predicates) would strengthen statistical validity. Evaluating multiple LLMs (GPT-4o, Claude 4, Gemini 2.5, open-source models such as DeepSeek-Coder) would quantify model-dependent effects on detection accuracy and rewrite quality. Testing with larger datasets (10K–1M rows) would measure whether detection accuracy degrades with data volume. Extending the execution engine layer to support Snowflake, BigQuery, Redshift, and Spark SQL would generalize the framework to production environments. Finally, studying the human-facing presentation of LLM-based query reviews—how to communicate risk scores, rewrite rationale, and validation outcomes to data engineers and analysts—would bridge the gap between automated monitoring and practical governance workflows under the 5V framework (volume, velocity, variety, veracity, value).

Reproducibility Statement

The complete framework, including all source code, dataset download scripts, workload definitions, prompt templates, analysis outputs, and documentation, is publicly available at [3]. The artifact is distributed with a README describing installation, execution, and reproduction of all results reported in this paper.

REFERENCES

- [1] M. Raasveldt and H. Muehleisen, “Duckdb: An embeddable analytical database,” in *Proc. Conf. Innovative Data Syst. Res. (CIDR)*, 2020.
- [2] SQLite Consortium, “Sqlite documentation,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.sqlite.org/docs.html>
- [3] S. Sharma, “Query monitoring framework,” GitHub repository, 2026, accessed: 2026-07-12. [Online]. Available: <https://github.com/shouvik-sharma/query-monitoring-framework>
- [4] “querylens: SQL inspection CLI,” GitHub, accessed: Jul. 2026. [Online]. Available: <https://github.com/querylens/querylens>
- [5] J. S. Lee, “sqlaudit: SQL query auditor and optimizer,” GitHub, 2026. [Online]. Available: <https://github.com/JSLEEKR/sqlaudit>
- [6] Google Cloud Platform, “Bigquery anti-pattern recognition,” GitHub, 2025. [Online]. Available: <https://github.com/GoogleCloudPlatform/bigquery-antipattern-recognition>
- [7] shirokurolab, “DWH-auditor: Bigquery cost optimization and data governance auditing,” GitHub, 2026. [Online]. Available: <https://github.com/shirokurolab/dwh-auditor>
- [8] Z. Sun, X. Zhou, G. Li, X. Yu, J. Feng, and Y. Zhang, “R-bot: An LLM-based query rewrite system,” *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5031–5044, 2025.
- [9] S. Zhou *et al.*, “LLMOpt: A query optimization method utilizing large language models,” 2025.
- [10] P. Akioyamen, Z. Yi, and R. Marcus, “The unreasonable effectiveness of LLMs for query optimization,” 2024.
- [11] S. Zhou *et al.*, “No more optimization rules: LLM-enabled policy-based multi-modal query optimizer,” 2024.
- [12] Y. Zhang *et al.*, “LLM4Hint: Leveraging large language models for hint recommendation in offline query optimization,” 2025.
- [13] Astronomer, “Snowpatrol: Anomaly detection and alerting for Snowflake usage,” GitHub, 2023. [Online]. Available: <https://github.com/astronomer/snowpatrol>
- [14] Keebo, “Making data clouds smarter: Automated warehouse optimization using data learning,” *Proc. ACM SIGMOD*, 2023.
- [15] S. Bress *et al.*, “Workload insights from the Snowflake data cloud: What do production analytic queries really look like?” *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5126–5139, 2025.

- [16] gundageren, “Patterns: AI-powered query pattern analysis,” GitHub, 2026. [Online]. Available: <https://github.com/gundageren/patterns>
- [17] Transaction Processing Performance Council, “TPC-H benchmark specification 3.0.1,” 2022. [Online]. Available: <https://www.tpc.org/tpch/>
- [18] —, “TPC-DS benchmark specification 4.0.0,” 2023. [Online]. Available: <https://www.tpc.org/tpcds/>
- [19] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?” *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015.
- [20] N. Huo *et al.*, “BIRD-INTERACT: Re-imagining Text-to-SQL evaluation for large language models via dynamic interactions,” 2025.
- [21] T. Yu, R. Zhang, K. Yang *et al.*, “SPIDER: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2018.
- [22] OpenAI, “GPT-4o mini documentation,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o-mini>
- [23] U.S. Geological Survey, “Earthquake catalog,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.usgs.gov/programs/earthquake-hazards>
- [24] NOAA NCEI, “Global hourly data,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.ncei.noaa.gov/>
- [25] AWID Dataset Mirror, “AWID intrusion detection system 2020,” GitHub, 2020, accessed: 2026-07-12. [Online]. Available: <https://github.com/krishanuskr/AWID-Intrusion-Detection-System-2020>